

Computer-Use Automation System

Design Report – interface.ai Take-Home Project

Dheeraj

September 2026

Contents

1. Architecture	2
Overview	2
Key Decisions	2
Trade-offs Accepted	2
2. Artifact Schema	3
Structure	3
Design Rationale	3
3. Determinism & Error Handling	4
How Replay is Deterministic	4
Error Taxonomy	4
Error Detection Flow (per step)	4
UI Drift	4
4. Heterogeneity & Multi-Tenant	5
Surface Abstraction	5
Multi-Tenant Reuse	5
5. Escalation & Handoff	6
Detecting “Stuck”	6
Control Transfer Model	6
6. Safety	7
Allowlist Enforcement	7
Action Risk Classification	7
PII Redaction	7
Known Limits	7
7. Cuts & Next Steps	8
What’s Implemented (thin but real)	8
What’s Mocked	8
What I’d Build Next	8
Repository	8

1. Architecture

Overview

The system has four layers, each with a clean boundary:

Layer	Role	Components
Entry	User/agent interface	CLI (Click), FastAPI server
Execution	Agent (discovery) + Replay engine (production)	Agent loop, Replay engine
Surface	Perception + Action abstraction	Browser (Playwright), Desktop (stub)
Cross-cutting	Schema, Safety, Escalation, Observability	Pydantic models, Allowlist, Handoff, Logger

Discovery path: CLI → Agent Loop → LLM Provider → Surface → Recorder → Artifact JSON

Production path: API/CLI → Replay Engine → Surface → Structured Result (*no LLM*)

Key Decisions

Python as the sole language. Pydantic gives typed schemas with JSON serialization for free. Playwright’s Python bindings are mature. FastAPI for the API surface. One language, one runtime, fewer integration seams.

Playwright + accessibility tree (hybrid observation). Playwright handles clicks and typing fast and reliably. The accessibility tree provides stable element identification even on hostile DOMs – and it’s the same concept available on desktop platforms via OS accessibility APIs (UIA, AXUIElement, AT-SPI). This is the credible bridge from web to desktop.

Single process, not microservices. Discovery agent, replay engine, and API server share a Python process and import the same schema types. No serialization boundary between them. Simpler to debug, deploy, and demo. The surface abstraction and artifact schema are the real boundaries.

Local mock target app. Full control over the surface and error states. Intentionally hostile markup (framesets, nested tables, no test IDs) directly demonstrates the legacy problem.

Trade-offs Accepted

- **No persistent database.** Artifacts are JSON files on disk. At scale → proper store needed.
- **Synchronous replay.** Each replay blocks until complete. At scale → job queue (Celery, etc.).
- **Single-browser sessions.** Each replay gets its own browser instance. At scale → pooling.

2. Artifact Schema

The artifact schema (`src/cua/schema/artifact.py`) is the centerpiece – a **typed, versioned, agent-invocable capability**.

Structure

Capability

```

+-- metadata      (name, version, surface_type, source_goal, tenant_id)
+-- inputs        (typed parameters the caller supplies)
+-- outputs       (typed fields extracted from the surface)
+-- steps[]       (ordered actions with full traceability)
|   +-- Step
|       +-- action + target      (what to do, where)
|       +-- pre_condition       (expected state before)
|       +-- post_condition      (expected state after)
|       +-- error_matchers      (patterns to check after action)
|       +-- retry config        (transient failure handling)
|       +-- risk_level          (safe / moderate / risky)
+-- success_condition (goal verification)
+-- error_handlers[] (global patterns at every step)

```

Design Rationale

Every step is a traceable node. Pre-condition → action → post-condition, with error matchers in between. On failure: which step, what was expected, what was observed, which pattern matched. No guessing.

Locator = single best + semantic descriptor. Each target element has one primary locator (typically a11y role/label) plus a human-readable semantic description. Primary drives replay; semantic aids debugging, review, and potential LLM-assisted recovery.

Template variables for parameterization. Steps reference inputs as `{{member_id}}`. Data flow is visible in the artifact JSON. Formal input contract (typed, with validation regex) means a calling agent knows exactly what to pass.

Three-tier error classification baked into the schema. Error matchers declare their category upfront: `business_outcome`, `recoverable`, or `hard_failure`. The replay engine doesn't re-classify at runtime.

Global error handlers. Session timeouts, permission denials, server errors can happen at any step. Global handlers checked at every step, sorted by priority.

Multi-tenant extension points. `tenant_id` and `parent_artifact_id` enable inheritance: a base artifact works for the common case, tenant-specific overrides only change the steps that differ.

3. Determinism & Error Handling

How Replay is Deterministic

1. **No LLM in the loop.** Every decision is encoded in the artifact.
2. **Template resolution.** `{{member_id}}` $\rightarrow$ concrete value before execution.
3. **Smart waits replace fixed sleeps.** Each checkpoint defines what “ready” looks like. The `SmartWaiter` polls with adaptive intervals (100ms $\rightarrow$ 2s backoff).
4. **Ordered, explicit steps.** Linear sequence in v1 – branching is the first extension.

Error Taxonomy

Category	Example	Response	Result
Business outcome	“Member not found”	Stop, return code	<code>{status: "business_outcome", outcome_code: "MEMBER_NOT_FOUND"}</code>
Recoverable	Session timeout dialog	Dismiss/retry/escalate	Logged, not surfaced unless exhausted
Hard failure	Element missing, 500 error	Stop, capture evidence	<code>{status: "failure", step_id, expected, observed}</code>

This separation is the most important design decision. “No such member” is a legitimate answer – not a crash.

Error Detection Flow (per step)

1. Check pre-condition (smart wait)
2. Scan global error handlers (before action)
3. Execute action (with retries on failure)
4. Scan step-level error matchers (after action)
5. Scan global error handlers (after action)
6. Check post-condition (smart wait)

UI Drift

The UIs are stable – drift is secondary. If a locator fails after retries, it’s a hard failure with a debuggable error. The semantic descriptor helps fix the locator without re-running discovery.

4. Heterogeneity & Multi-Tenant

Surface Abstraction

The **Surface** abstract class defines the interface between automation and any UI:

- **Observation:** `observe()` $\rightarrow$ `SurfaceState` (URL, text, a11y tree, screenshot)
- **Action:** `click()`, `type_text()`, `navigate()`, etc.
- **Session mgmt:** `pause()`, `resume()`, `is_alive()`

The browser implementation uses Playwright for actions and `aria_snapshot()` for observation. A desktop implementation would use platform accessibility APIs.

Why accessibility tree? It's the only approach that works consistently across web, legacy web, and desktop. Browsers expose it. Operating systems expose it. Screen readers depend on it. Locators based on a11y roles/names translate directly to desktop queries.

The seam: The artifact stores locators in surface-agnostic terms. The Surface translates them. New surface = new implementation, not schema changes.

Multi-Tenant Reuse

Two mechanisms in the schema:

1. **tenant_id + parent_artifact_id:** Base artifact (tenant_id=null) for the common flow. Tenant overrides only change differing steps.
2. **URL pattern parameterization:** Recorder normalizes routes into patterns (`/member/M1001` $\rightarrow$ `/member/{{member_id}}`).

Drift detection (design): Compare a11y tree snapshots from recording with current trees during replay. Flag stale artifacts per tenant via scheduled health checks.

5. Escalation & Handoff

Detecting “Stuck”

The StuckDetector tracks:

1. **No progress:** URL and content unchanged after N actions (hashing-based)
2. **Repeated failure:** Same step failed > N times
3. **Unknown state:** No error matcher or condition matches
4. **Risky action:** Irreversible action without `confirm_risky` flag

Control Transfer Model

AUTOMATION running

```
|
+-- stuck/error detected
|
v
```

PAUSE (session_controller = "human")

```
|
+-- InterventionRequest: reason, step, screenshot, noVNC URL
|
v
```

HUMAN operates the SAME browser session

```
|
+-- "Resume" via operator API
|
v
```

RESUME (session_controller = "automation")

```
+-- Re-observe state, continue
+-- Log intervention window
```

Key invariant: Automation NEVER acts while `session_controller == "human"`. Changed only by explicit API calls.

What’s real: Pause/resume mechanism, intervention request with context, `session_controller` flag, operator API.

What’s mocked: Operator UI (minimal HTML page with noVNC link and Resume button).

6. Safety

Allowlist Enforcement

Every URL and action checked against `config/allowlist.yml`:

- Permitted domains (regex) – only localhost by default
- Allowed action types per domain
- Blocked paths (regex)
- Per-run action caps

Action Risk Classification

Level	Actions	Policy
Safe	navigate, extract, assert, screenshot	Always allowed
Moderate	type, clear, select, hover	Allowed, logged
Risky	click on submit/confirm/delete/transfer	Blocked in discovery; requires <code>confirm_risky</code> or escalation in replay

Classification uses action type + contextual signals (button text, URL patterns).

PII Redaction

- `sensitive: true` parameters -> [REDACTED] in all logs and evidence
- Known PII patterns (SSN, account numbers, phone, email) auto-detected
- Sensitive values tracked per-run for complete redaction

Known Limits

- Domain-based allowlist, not element-based
- Screenshots not redacted (region-based blurring is a future enhancement)
- PII patterns are US-centric

7. Cuts & Next Steps

What's Implemented (thin but real)

Capability	Status
Goal-driven agent loop (LLM)	Real – observe -> decide -> act
Structured artifact	Real – full Pydantic schema, versioned JSON
Deterministic replay	Real – no LLM, smart waits, error scanning
Three-tier error handling	Real – business_outcome / recoverable / hard_failure
Human-in-the-loop escalation	Real mechanism – pause/resume/control transfer
Safety guardrails	Real – allowlist, classification, PII redaction
API server (agent-invocable)	Real – FastAPI with capabilities catalog
Target app (legacy surface)	Real – hostile markup, injectable errors
LLM providers	Real – Anthropic, OpenAI, and Google Gemini adapters

What's Mocked

- **Operator console:** Minimal HTML. Real mechanism, mock presentation.
- **Desktop surface:** Design stub with documented extension notes.
- **noVNC integration:** Docker config provided, needs environment-specific setup.

What I'd Build Next

1. **Conditional branching** – condition field on steps with expression evaluator
2. **LLM-assisted single-step recovery** – bounded, policy-checked, recorded as evidence
3. **Screenshot region redaction** – bounding-box-based blurring for sensitive data
4. **Cross-tenant artifact inheritance** – base/override merge logic + drift detection
5. **Approval workflow** – draft -> approved, confidence scoring from replay success rate
6. **Parallel stability testing** – replay N times, report flakiness signal

Repository

GitHub: <https://github.com/dheeraj7000/interfaceai-takehome>

All evidence (discovery run, replay runs, artifacts) included in `/evidence/` and `/artifacts/`.